

Solutioning Product Service — Vela

Spec for the dev homework test (see dev-homework.md). **Vela** is a (fictional) custom keyboard brand. Design assets in vela-assets/, seed data in the Appendix. This document follows DF Labs' standard solutioning format — and doubles as an example of the documents you will receive in real work.

A. BACKGROUND

The Product Service is the central point for managing product data in the Vela system. Vela is a custom keyboard brand selling assembled keyboards, switches, keycaps, and accessories. The Product Service is the main component providing the product data shown in the catalog.

In the Vela project, the Product Service will:

- Store product data (name, sku, category, price, stock, active status, description).
- Provide the **GET API** consumed by the catalog page, including search and category filtering.
- Be the single consistent source of product data for every view.

B. PROBLEM STATEMENT

Today Vela's product catalog is just Instagram posts, and the product list is tracked manually in a spreadsheet. This causes several problems:

- Potential buyers have no single place to see all products with prices and availability.
- Product data is scattered and inconsistent across records.
- The catalog's presentation does not reflect Vela's positioning as a premium brand.

A Product Service is required that can:

- Provide centralized, consistent product data through one API.
- Support a catalog page that matches Vela's visual identity.

C. PROPOSED SOLUTION

User Flow (CUSTOMER): opens the catalog page -> the page calls the product GET API -> Vela's products appear (image, name, category, price, availability) -> searches by name or filters by category (keyboard/switch/kit) -> the list refreshes from the same API.

(In a real project this section is a flowchart.)

Architecture — the technical rules devs will follow during the build:

- **Stack:** free by agreement (e.g. Laravel or Next.js full-stack) — one monolith application.
- **Layering pattern:** route/page -> controller/handler -> service (logic + queries) -> model. Validation has one home (from request / service) — never scattered across views.
- **Database:** PostgreSQL recommended (SQLite acceptable locally); data access only through models/ORM.
- **Notes for devs:** no logic in views; configuration (database connection, etc.) through env, never hardcoded.

D. DESIGN

The visual anchor lives in **vela-assets/**:

- design-page.png — **the official Vela page design** (nav, typography, color, layout rhythm). The catalog page must follow this design language — it should look like another page of the same website: **dark, premium, minimal**.
 - product-hero/top/detail/exploded.png — official product renders, used as the product images.
1. Product Catalog page — anchor: vela-assets/design-page.png

E. API SPECIFICATION

GET All Product	
Description	Used to fetch the list of Vela products consumed by the catalog page. The API uses request filters, so a single get-all API also covers get-by-field.
Endpoint	GET /product-service/products
Param	search (name/sku), category
Request Header	-
Request Payload	-
Status	DEV
Response Header	200
Response Body	{ "content": [{ "id": 1, "name": "Vela V1 75%", "sku": "VLA-K75", "category": "keyboard", "price": 1850000, "stock": 12, "active": true, "image": "product-hero.png", "description": "..." }] }

F. TO-DO LIST

No	User Story	Day	PIC
1.	Create Table for Product + seeder from the Appendix [BE]	1	@...
2.	Create API GET All Product (search + category filter) [BE]	1	@...
3.	Create Catalog page (consumes the API, anchor vela-assets/) [FE]	1	@...

G. QUALITY PASS

Filled by the developer when a story is done — no evidence, no review, and the story does not count as done:

No	User Story	Required evidence	Evidence (filled by dev)	Date
1.	Product table + seeder [BE]	Successful migrate + seed output; the 4 Appendix products present in the database		
2.	API GET All Product [BE]	Full sample response + filtered responses (?search=, ?category=)		

No	User Story	Required evidence	Evidence (filled by dev)	Date
3.	Catalog page [FE]	Catalog screenshot + search/filter results, compared against the anchor		

Appendix — Seed Data

Used to seed the database (PostgreSQL recommended). JSON format, free to convert into a seeder/SQL inserts. The image column points to files in vela-assets/ — each product has its own image:

```
[
  { "name": "Vela V1 75%", "sku": "VLA-K75", "category": "keyboard",
    "price": 1850000, "stock": 12, "active": true, "image": "product-hero.png", "description":
    "75% hot-swap keyboard, CNC aluminium case, volume knob." },
  { "name": "Vela V1 Low-Profile", "sku": "VLA-K75-LP", "category": "keyboard",
    "price": 1650000, "stock": 8, "active": true, "image": "product-top.png", "description":
    "Low-profile variant for light typing, same 75% layout." },
  { "name": "Vela Switch Tactile 45g (10 pcs)", "sku": "VLA-SW-T45", "category": "switch",
    "price": 85000, "stock": 120, "active": true, "image": "product-detail.png", "description":
    "45g tactile switches, factory lubed, 3-pin MX-style." },
  { "name": "Vela V1 Barebones Kit", "sku": "VLA-K75-BB", "category": "kit",
    "price": 1250000, "stock": 6, "active": true, "image": "product-exploded.png", "description":
    "Case, plate, gasket, and hot-swap PCB – build with your choice of switches and keycaps." }
]
```